Visualizing *n*-Dimensional Virtual Worlds with *n*-Vision

Steven Feiner
Clifford Beshers

Department of Computer Science
Columbia University
New York, New York 10027

1 Introduction

There are many applications in science, mathematics, statistics, and business, in which it is important to explore and manipulate data in more than three dimensions. In these applications, data can be defined by points in Euclidean *n*-space. A point's position is then specified with *n* coordinates, each of which determines its position relative to one of *n* mutually perpendicular axes. We describe here research that has as its goal the development of interaction techniques and metaphors for the 4D and higher-dimensional worlds that this data represents.

2 The *n*-Vision Testbed

n-Vision is a testbed for exploring *n*-dimensional virtual worlds. It relies, in part, on techniques for transforming and displaying 4D objects in real-time using 3D graphics hardware, as described in [BESH88]. The system supports techniques such as ''near'' and ''far'' hyperplane clipping, and shading (including the use of color to accomplish 4D depth cueing, in which the intensity is a function of the fourth, *w* coordinate).

Objects in *n*-Vision may be manipulated through mouse-operated control panels, conventional dial and knob boxes, and animation scripts. In addition, we are exploring the use of an inherently 3D interaction device, the VPL DataGlove [ZIMM87]. The DataGlove uses magnetic sensors to monitor the 3D position and orientation of the user's hand, and fiber optic cables running along each finger to monitor two joint angles for each of the five fingers. Software supports simple recognition of gestures defined by ranges of joint angles. A stereo display system allows us to view and manipulate worlds in 3D [STER89].

3 *n*-Dimensional Interaction Metaphors

Perhaps the simplest way to control the *n* values that define a point in *n*-space is to assign each value to its own valuator device. For example, a knob box may control as many independent variables as it has knobs. Similarly, the values of each of these *n* variables may be displayed by assigning each to its own numeric string or one-dimensional variable-length bar. In 2D and 3D applications, we commonly take advantage of our experience manipulating and viewing real objects, and instead use interaction and display devices that treat these virtual objects as if they were part of our 3D world. Our spatial positioning experience, however, is inherently limited to 3D. Therefore, from the standpoint of the user interface, the straightforward generalization of specifying a point in 2D by pointing in a plane, to specifying a point in 3D by pointing in space, does not extend to higher dimensions. What can we do instead?

We describe some of the techniques that we are developing in context of a specific application in what may be called ''financial visualization.'' In this application, the user is interested in exploring the value of options to buy or sell foreign currency on a specified date at a specified price. These are called *European call options* or *put options*, respectively, and their value is modeled as a function of six variables: the price at which the currency can be bought or sold at maturity (''strike'' price), the price at which it is selling now (''spot'' price), the time remaining to the date at which the option must be exercised, the domestic and foreign interest rates, and the volatility of the price [HULL89]. This function of six variables defines a surface in 7-space.

3.1 Worlds within Worlds

One common approach to reducing the complexity of a multivariate function is to hold one or more of its independent variables constant. Each constant corresponds to taking an infinitely thin slice of the world perpendicular to the constant variable's axis, reducing the world's dimension. If we reduce the dimension to 3D (a height-field function of two variables), the resulting slice can be manipulated and displayed using conventional 3D graphics hardware. Although this simple approach effectively slices away the higher dimensions, it is possible to add them back. Imagine embedding the 3D world in another 3D coordinate system. The position of the inner 3D world (as defined by its origin) relative to the containing coordinate system can specify the values of three of the variables that were held constant by slicing the world down to size. Additional variables can be supported through further recursive nesting.

We allow the user to create these ''worlds within worlds'' by specifying the assignment of variables to coordinate systems and their axes. They can then use the DataGlove to ''grab'' a world (through gesture recognition) and translate it relative to the coordinate system in which it is embedded. Moving a world relative to the world in which it is embedded will change up to three otherwise constant variables, which causes the object(s) within the world to change.

We can deposit multiple copies of the same world within the containing world, to allow the copies to be compared visually. Each copy has a different constant set of values of the containing world's variables. By clipping each world to a finite volume, we can move each copy arbitrarily close to another without overlap. 3D transformations allow the worlds to be rotated and scaled, and viewed from different positions. Figure 1 shows a stereo pair of a surface that represents the value of a ''butterfly spread,'' a trading strategy that involves buying and selling call options for the same currency and maturity date, visualized as a function of five

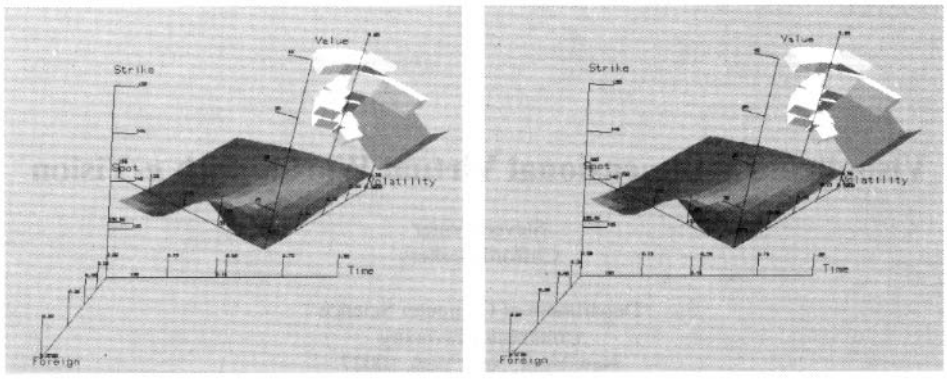

Figure 1 Stereo pair of currency option spread in nested worlds.

variables (the domestic interest rate has been held constant, and is not shown as an axis). The outer world has axes of time to maturity, strike price, and foreign interest. A single inner world plots the value of the spread as a function of spot price and volatility. Tick marks on the outer axes mark the position of the inner world's origin. In this example, the strike price axis is used to control only some of the options in the spread, while the others remain fixed. A "dipstick," shown here at the rear of the surface, can be moved to read the value at the point at which it intersects the surface.

3.2 Hyperworlds Within Hyperworlds

Rather than limiting our worlds to 3D, they can be of higher-dimension instead. In this case, each "hyperworld" may be transformed first in its original dimension before being sliced and projected down to 3D, and finally 2D. The user may interact with each hyperworld as a 3D world, and may also use the DataGlove to manipulate the hyperworld in its original dimensionality. Our implementation currently supports hyperworlds of 4D only. Figure 2 shows a 3D outer world whose axes are domestic interest rate, strike price, and foreign interest rate. The inner 4D hyperworld plots the value of a put option as a function of spot price, time to maturity, and volatility. Allowing arbitrary 4D transformations and projections makes it possible for the user to explore, searching for interesting properties that might otherwise go unnoticed if the only dimension-reducing operations supported were slicing or orthographic projection perpendicular to an axis.

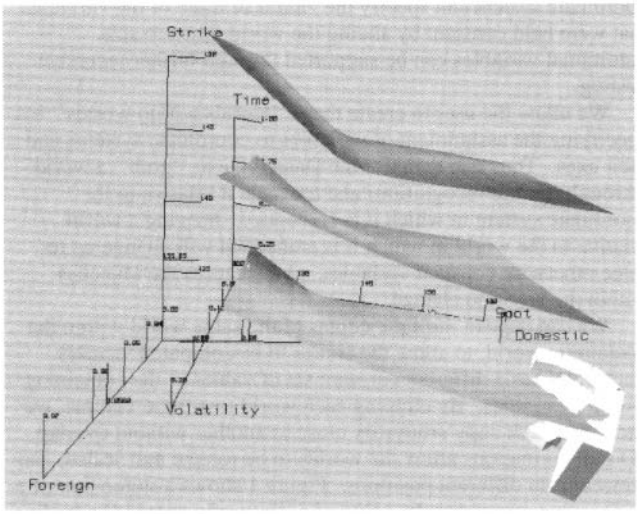

Figure 2 Value of a single put option in 4D.

4 Implementation

n-Vision is implemented in C++ and runs under UNIX on an Hewlett-Packard 9000 370 TurboSRX workstation, which has hardware support for scan-conversion, shading, and *z*-buffer hidden-surface removal. It takes approximately 1/4 second each to generate the views in Figures 1 and 2.

5 Future Work

Currently, the user decides the techniques that should be used to present the world. We are interfacing *n*-Vision to Scope [BESH89], a rule-based system that designs user interface control panels by choosing and laying out appropriate widgets. Scope's rule base is being expanded to include rules for assigning variables to the DataGlove's valuators, taking into account their interdependencies, and the relative difficulty of using hands and fingers to control variables. Scope is implemented in C++ and the production system language CLIPS [GIAR88].

6 Acknowledgments

This work is supported in part by a gift from Citicorp, by the Hewlett-Packard Company under its AI University Grants Program, and by the New York State Center for Advanced Technology under Contract NYSSTF-CAT(89)-5. We wish to thank Dan Schutzer for sharing his financial expertise and David Sturman for providing us with drivers for the VPL DataGlove.

7 References

[BESH88] Beshers, C. and Feiner, S. "Real-Time 4D Animation on a 3d Graphics Workstation." *Proc. Graphics Interface '88*, Edmonton, June 6–10, 1988, 1–7.

[BESH89] Beshers, C., and Feiner, S. "Scope: Automated Generation of Graphical Interfaces." *Proc. ACM UIST '89*, Williamsburg, VA, November 13–15, 1989, 76–85.

[GIAR88] Giarratano, J. *CLIPS User's Guide*. Artificial Intelligence Section/NASA, Lyndon B. Johnson Space Center, TX, 1988.

[HULL89] Hull, J. *Options, Futures, and Other Derivative Securities*, Prentice-Hall, NJ, 1989.

[STER89] StereoGraphics, CrystalEyes product literature, StereoGraphics Corp., San Rafael, CA, 1989.

[ZELT89] Zeltzer, D., Pieper, S., and Sturman, D. "An Integrated Graphical Simulation Platform." *Proc. Graphics Interface '89*, London, Ontario, June 19–23, 1989, 266–274.

[ZIMM87] Zimmerman, T., Lanier, J., Blanchard, C., Bryson, S., and Harvill, Y. "A Hand Gesture Interface Device." *Proc. CHI + GI 1987*, Toronto, Ontario, April 5–7, 1987, 189–192.